

ER SCHEMA USER VENDOR FLOW AND ATTRIBUTES

USER FLOW

PART 1 — Registration & Login

SCREEN: Sign Up

User fills:

- full_name
- email
- phone
- date_of_birth
- home_city

System creates one User row.

User Entity — All Attributes:

user_id	SERIAL PRIMARY KEY	
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(120)	UNIQUE NOT NULL
phone	VARCHAR(20)	UNIQUE NOT NULL
date_of_birth	DATE	NOT NULL
home_city	VARCHAR(100)	
account_status	VARCHAR(20)	DEFAULT 'ACTIVE' CHECK (ACTIVE/SUSPENDED/DELETED)
created_at	TIMESTAMP	DEFAULT NOW()

PART 2A — Transport Booking Flow

Stage 1 — User SELECTS ScheduleDeparture

(First half of Aggregation Box 1)

Search Screen

SCREEN: Search Transport

User enters:

- transport_mode (FLIGHT/TRAIN/BUS)
- origin_city
- destination_city
- travel_date
- number of seats

System queries through these tables in order:

TransportRoute

- match origin_city + destination_city + transport_mode

RouteSchedule

- match route_id
- vendor_id tells us which airline/railway
- check days_of_week includes travel_date's weekday

ScheduleClass

- get all classes under matching schedules

ScheduleDeparture

- match schedule_id + class_id
- WHERE departure_date = travel_date
- WHERE available_seats >= requested seats
- WHERE status = SCHEDULED

TransportRoute — All Attributes:

route_id	SERIAL PRIMARY KEY	
transport_mode	VARCHAR(20)	NOT NULL CHECK (FLIGHT/TRAIN/BUS)
origin_city	VARCHAR(100)	NOT NULL
destination_city	VARCHAR(100)	NOT NULL
vehicle_number	VARCHAR(50)	

RouteSchedule — All Attributes:

schedule_id	SERIAL PRIMARY KEY	
route_id	INT	FK → TransportRoute
vendor_id	INT	FK → Vendor
departure_time	TIME	NOT NULL
arrival_time	TIME	NOT NULL
days_of_week	VARCHAR(50)	(MON,WED,FRI)
is_active	BOOLEAN	DEFAULT TRUE

ScheduleClass — All Attributes:

class_id	SERIAL PRIMARY KEY	
schedule_id	INT	FK → RouteSchedule
class_name	VARCHAR(50)	(ECONOMY/BUSINESS/ SLEEPER/3AC/2AC/1AC)

total_seats	INT	NOT NULL
base_fare	DECIMAL(10,2)	NOT NULL

ScheduleDeparture — All Attributes:

departure_id	SERIAL PRIMARY KEY	
schedule_id	INT	FK → RouteSchedule
class_id	INT	FK → ScheduleClass
departure_date	DATE	NOT NULL
available_seats	INT	NOT NULL
dynamic_fare	DECIMAL(10,2)	NOT NULL
status	VARCHAR(20)	DEFAULT 'SCHEDULED'
		CHECK (SCHEDULED/ CANCELLED/COMPLETED)

Results Screen

SCREEN: Available Options

User sees per card:

- vehicle_number (from RouteSchedule via TransportRoute)
- origin_city
- destination_city
- departure_time
- arrival_time
- class_name
- available_seats
- dynamic_fare ← actual price user pays
- DemandTag badge (FILLING FAST / HIGH DEMAND)

User picks one card → one `ScheduleDeparture` row selected.

This completes the **selects** relationship in Aggregation Box 1.

Stage 2 — confirms → TransportBooking created

(Second half of Aggregation Box 1)

Passenger Details Screen

SCREEN: Enter Passenger Details

For each seat, user fills:

- full_name
- age
- gender
- govt_id_type
- govt_id_number

Passenger — All Attributes:

passenger_id	SERIAL PRIMARY KEY	
booking_id	INT	FK → TransportBooking
full_name	VARCHAR(100)	NOT NULL
age	INT	NOT NULL
gender	VARCHAR(10)	CHECK (MALE/FEMALE/OTHER)
govt_id_type	VARCHAR(20)	CHECK (AADHAR/PASSPORT/PAN)
govt_id_number	VARCHAR(50)	NOT NULL
seat_number	VARCHAR(10)	

Promo + Payment Screen

SCREEN: Review & Pay

User sees:

- dynamic_fare × num_seats = base_amount
- enters promo code optionally
- selects payment_method
- pays final_amount

System checks PromoCode:

- code exists and is_active = TRUE
- valid_from ≤ TODAY ≤ valid_to
- applicable_on = TRANSPORT or ALL
- min_booking_amount ≤ base_amount
- used_count < max_uses

PromoCode — All Attributes:

promo_id	SERIAL PRIMARY KEY	
code	VARCHAR(50)	UNIQUE NOT NULL
discount_type	VARCHAR(20)	CHECK (FLAT/PERCENTAGE)
discount_value	DECIMAL(10,2)	NOT NULL
max_discount_cap	DECIMAL(10,2)	
min_booking_amount	DECIMAL(10,2)	
valid_from	DATE	NOT NULL
valid_to	DATE	NOT NULL
applicable_on	VARCHAR(20)	CHECK (TRANSPORT/ HOTEL/PACKAGE/ALL)
max_uses	INT	
used_count	INT	DEFAULT 0
is_active	BOOLEAN	DEFAULT TRUE

On payment success, system creates:

Booking — All Attributes:

booking_id	SERIAL PRIMARY KEY	
user_id	INT	FK → User NOT NULL
booking_type	VARCHAR(20)	NOT NULL
		CHECK (TRANSPORT/HOTEL/PACKAGE)
booked_at	TIMESTAMP	DEFAULT NOW()
status	VARCHAR(20)	DEFAULT 'CONFIRMED'

		CHECK (CONFIRMED/ CANCELLED/COMPLETED)
base_amount	DECIMAL(10,2)	NOT NULL
discount_amount	DECIMAL(10,2)	DEFAULT 0
final_amount	DECIMAL(10,2)	NOT NULL
promo_id	INT	FK → PromoCode NULLABLE
payment_method	VARCHAR(20)	CHECK (UPI/CARD/ NETBANKING/WALLET)
payment_ref	VARCHAR(100)	

TransportBooking — All Attributes:

booking_id	INT PRIMARY KEY	FK → Booking
departure_id	INT	FK → ScheduleDeparture NOT NULL
num_seats	INT	NOT NULL

Then simultaneously:

ScheduleDeparture:
→ $\text{available_seats} = \text{available_seats} - \text{num_seats}$

PromoCode:
→ $\text{used_count} = \text{used_count} + 1$

Confirmation Screen

SCREEN: Booking Confirmed ✓

User sees:

- booking_id (their PNR)
- vehicle_number
- origin → destination
- departure_date + departure_time
- class_name
- passenger names + seat numbers
- final_amount paid
- payment_ref
- "Add Travel Insurance?" prompt



PART 2B — Hotel Booking Flow

Stage 1 — User BROWSES RoomCategory

(First half of Aggregation Box 2)

Search Screen

SCREEN: Search Hotels

User enters:

- city
- checkin_date
- checkout_date
- num_rooms
- num_guests

System queries:

Hotel

- WHERE city matches
- WHERE is_active = TRUE

RoomCategory

- WHERE hotel_id matches
- WHERE max_guests >= num_guests

RoomAvailability

- WHERE room_category_id matches
- WHERE available_date BETWEEN
checkin_date AND checkout_date - 1
- WHERE available_rooms >= num_rooms
- EVERY date in range must pass this check

Hotel — All Attributes:

hotel_id	SERIAL PRIMARY KEY		
vendor_id	INT	FK → Vendor	NOT NULL
hotel_name	VARCHAR(150)	NOT NULL	
city	VARCHAR(100)	NOT NULL	
full_address	TEXT		
star_rating	INT	CHECK (1-5)	
amenities	TEXT		
checkin_time	TIME		
checkout_time	TIME		
is_active	BOOLEAN	DEFAULT TRUE	

RoomCategory — All Attributes:

room_category_id	SERIAL PRIMARY KEY		
hotel_id	INT	FK → Hotel	NOT NULL
category_name	VARCHAR(100)	NOT NULL	
		(STANDARD/DELUXE/SUITE)	
max_guests	INT	NOT NULL	
price_per_night	DECIMAL(10,2)	NOT NULL	
total_rooms	INT	NOT NULL	

RoomAvailability — All Attributes:

room_category_id	INT	FK → RoomCategory	}
available_date	DATE	NOT NULL	} COMPOSITE PK
available_rooms	INT	NOT NULL	

Results Screen

SCREEN: Available Hotels + Rooms

Hotel card shows:

- hotel_name
- star_rating
- amenities
- checkin_time / checkout_time

Room card shows:

- category_name
- max_guests
- price_per_night
- available_rooms for their date range
- DemandTag badge
- total_price = price_per_night × num_rooms × num_nights

User picks one room category.

This completes the **browses** relationship in Aggregation Box 2.

Stage 2 — confirms → HotelBooking created

(Second half of Aggregation Box 2)

Guest Details Screen

SCREEN: Enter Guest Details

For each room, user fills:

- guest_name (lead guest)

HotelGuest — All Attributes:

guest_id	SERIAL PRIMARY KEY	
booking_id	INT	FK → HotelBooking NOT NULL
guest_name	VARCHAR(100)	NOT NULL

Promo + Payment Screen

- base_amount = price_per_night × num_rooms × num_nights
- promo applied if valid
- final_amount calculated
- payment processed

System creates:

HotelBooking — All Attributes:

booking_id	INT PRIMARY KEY	FK → Booking	
hotel_id	INT	FK → Hotel	NOT NULL

room_category_id	INT	FK → RoomCategory	NOT NULL
checkin_date	DATE	NOT NULL	
checkout_date	DATE	NOT NULL	
num_rooms	INT	NOT NULL	
num_guests	INT	NOT NULL	

Then simultaneously:

RoomAvailability:
→ available_rooms DECREMENTED
→ for EACH date between checkin_date and checkout_date - 1

PART 2C — Package Booking Flow

Stage 1 — User EXPLORES PackageDeparture

(First half of Aggregation Box 3)

Browse Screen

SCREEN: Browse Packages

User filters by:

→ theme
→ duration_days
→ budget

Package card shows:

→ package_name
→ theme
→ destinations
→ duration_days
→ inclusions
→ base_price_per_person

HolidayPackage — All Attributes:

package_id	SERIAL PRIMARY KEY		
vendor_id	INT	FK → Vendor	NOT NULL
package_name	VARCHAR(150)	NOT NULL	
theme	VARCHAR(100)	CHECK (BEACH/ADVENTURE/ HONEYMOON/FAMILY/ PILGRIMAGE)	
destinations	TEXT	NOT NULL	
duration_days	INT	NOT NULL	
duration_nights	INT	NOT NULL	
inclusions	TEXT		
base_price_per_person	DECIMAL(10,2)	NOT NULL	
is_active	BOOLEAN	DEFAULT TRUE	

Departure Selection Screen

SCREEN: Select Departure Slot

User sees slots for chosen package:

- departure_date
- available_seats
- price_per_person (may differ from base price)
- status badge (OPEN / FILLING FAST / FULL)

PackageDeparture — All Attributes:

package_departure_id	SERIAL PRIMARY KEY	
package_id	INT	FK → HolidayPackage NOT NULL
departure_date	DATE	NOT NULL
total_seats	INT	NOT NULL
available_seats	INT	NOT NULL
price_per_person	DECIMAL(10,2)	NOT NULL
status	VARCHAR(20)	DEFAULT 'OPEN' CHECK (OPEN/FULL/CANCELLED)

User picks one slot.

This completes the **explores** relationship in Aggregation Box 3.

Stage 2 — confirms → PackageBooking created

(Second half of Aggregation Box 3)

Traveller Details Screen

SCREEN: Enter Traveller Details

For each traveller:

- full_name
- age
- gender
- govt_id_type
- govt_id_number

PackageTraveller — All Attributes:

package_traveller_id	SERIAL PRIMARY KEY	
booking_id	INT	FK → PackageBooking NOT NULL
full_name	VARCHAR(100)	NOT NULL
age	INT	NOT NULL
gender	VARCHAR(10)	CHECK (MALE/FEMALE/OTHER)
govt_id_type	VARCHAR(20)	CHECK (AADHAR/PASSPORT/PAN)
govt_id_number	VARCHAR(50)	NOT NULL

PackageBooking — All Attributes:

booking_id	INT PRIMARY KEY	FK → Booking
package_departure_id	INT	FK → PackageDeparture NOT NULL
num_travellers	INT	NOT NULL

Then simultaneously:

PackageDeparture:

- available_seats = available_seats - num_travellers
- IF available_seats = 0 THEN status = FULL

PART 3 — Post Booking Features

Insurance

SCREEN: Add Insurance (after confirmation screen)

User sees plan options and picks one.
Connects directly to Booking (1:1).

Insurance — All Attributes:

insurance_id	SERIAL PRIMARY KEY		
booking_id	INT	FK → Booking	UNIQUE NOT NULL
provider_name	VARCHAR(150)	NOT NULL	
plan_name	VARCHAR(150)	NOT NULL	
plan_type	VARCHAR(20)	CHECK (SINGLE_TRIP/MULTI_TRIP)	
coverage_summary	TEXT		
premium_amount	DECIMAL(10,2)	NOT NULL	
valid_from	DATE	NOT NULL	
valid_to	DATE	NOT NULL	
claim_status	VARCHAR(20)	DEFAULT 'NOT_CLAIMED'	
		CHECK (NOT_CLAIMED/	
		CLAIM_FILED/SETTLED)	

Cancellation & Refund

SCREEN: My Bookings → Cancel

User selects booking and cancels.
Connects directly to Booking (1:1).

CancellationRefund — All Attributes:

cancellation_id	SERIAL PRIMARY KEY		
booking_id	INT	FK → Booking	UNIQUE NOT NULL
cancelled_at	TIMESTAMP	DEFAULT NOW()	
reason	TEXT		
refund_amount	DECIMAL(10,2)		

refund_status	VARCHAR(20)	DEFAULT 'PENDING'
		CHECK (PENDING/
		PROCESSED/REJECTED)
processed_at	DATE	

On cancellation:

Booking.status → CANCELLED

ScheduleDeparture.available_seats	INCREMENTED	(if transport)
RoomAvailability.available_rooms	INCREMENTED	(if hotel, per date)
PackageDeparture.available_seats	INCREMENTED	(if package)
PackageDeparture.status	→ OPEN	(if was FULL)

Review

SCREEN: Rate Your Experience
(only appears when Booking.status = COMPLETED)

User fills:
→ star_rating
→ title
→ comment

Connects directly to Booking (1:1).
booking_id proves they actually travelled.

Review — All Attributes:

review_id	SERIAL PRIMARY KEY		
booking_id	INT	FK → Booking	UNIQUE NOT NULL
user_id	INT	FK → User	NOT NULL
entity_type	VARCHAR(20)	CHECK (TRANSPORT/HOTEL/PACKAGE)	
entity_id	INT	NOT NULL	
star_rating	INT	CHECK (1-5)	NOT NULL
title	VARCHAR(150)		
comment	TEXT		
reviewed_at	DATE	DEFAULT CURRENT_DATE	

DemandTag — Background Job

No user interaction. System computed.
Background job runs periodically and checks:

ScheduleDeparture.available_seats	vs	total_seats
RoomAvailability.available_rooms	vs	total_rooms
PackageDeparture.available_seats	vs	total_seats

Writes tags accordingly.

DemandTag — All Attributes:

tag_id	SERIAL PRIMARY KEY	
service_type	VARCHAR(20)	CHECK (TRANSPORT/HOTEL/PACKAGE)
service_id	INT	(loose ref – no FK constraint)
tag_value	VARCHAR(30)	CHECK (HIGH_DEMAND/ FILLING_FAST/ LOW_AVAILABILITY)
last_computed	TIMESTAMP	DEFAULT NOW()



VENDOR FLOW

Step 1 — Vendor Registration

SCREEN: Register as Vendor

Vendor fills:

- company_name
- vendor_type
- gst_number
- registered_address
- contact details

Vendor — All Attributes:

vendor_id	SERIAL PRIMARY KEY	
company_name	VARCHAR(150)	NOT NULL
vendor_type	VARCHAR(20)	NOT NULL
		CHECK (TRANSPORT/ HOTEL/PACKAGE)
gst_number	VARCHAR(50)	UNIQUE NOT NULL
registered_address	TEXT	NOT NULL
website	VARCHAR(150)	
contact_name	VARCHAR(100)	NOT NULL
contact_phone	VARCHAR(20)	
contact_email	VARCHAR(120)	
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()

Step 2A — Transport Vendor Lists Services

SCREEN 1: Add Route

- transport_mode
- origin_city
- destination_city
- vehicle_number

→ Creates TransportRoute row

SCREEN 2: Add Schedule to Route

→ departure_time
→ arrival_time
→ days_of_week
→ Creates RouteSchedule row (with vendor_id here)

SCREEN 3: Add Classes

→ class_name
→ total_seats
→ base_fare
→ Creates ScheduleClass rows

System AUTO-GENERATES ScheduleDeparture rows:
→ for next 365 days
→ for each (schedule_id × class_id) combination
→ available_seats = total_seats
→ dynamic_fare = base_fare
→ status = SCHEDULED

Step 2B — Hotel Vendor Lists Property

SCREEN 1: Add Hotel

→ hotel_name, city, full_address
→ star_rating, amenities
→ checkin_time, checkout_time
→ Creates Hotel row

SCREEN 2: Add Room Categories

→ category_name
→ max_guests
→ price_per_night
→ total_rooms
→ Creates RoomCategory rows

System AUTO-GENERATES RoomAvailability rows:
→ for next 365 days
→ available_rooms = total_rooms for every date

Step 2C — Package Vendor Lists Package

SCREEN 1: Create Package

→ package_name, theme

- destinations
- duration_days, duration_nights
- inclusions
- base_price_per_person
- Creates HolidayPackage row

SCREEN 2: Add Departure Slots

- departure_date
- total_seats
- price_per_person
- Creates PackageDeparture rows
- available_seats = total_seats
- status = OPEN

Complete Schema at a Glance

23 TABLES TOTAL

User	→ 8 attributes	
Vendor	→ 10 attributes	
TransportRoute	→ 5 attributes	
RouteSchedule	→ 7 attributes	(vendor_id lives here)
ScheduleClass	→ 5 attributes	(weak)
ScheduleDeparture	→ 7 attributes	(weak)
Hotel	→ 10 attributes	
RoomCategory	→ 6 attributes	(weak)
RoomAvailability	→ 3 attributes	(weak, composite PK)
HolidayPackage	→ 10 attributes	
PackageDeparture	→ 7 attributes	(weak)
Booking	→ 11 attributes	(superclass)
TransportBooking	→ 3 attributes	(subclass)
HotelBooking	→ 7 attributes	(subclass)
PackageBooking	→ 3 attributes	(subclass)
Passenger	→ 8 attributes	(weak under TransportBooking)
HotelGuest	→ 3 attributes	(weak under HotelBooking)
PackageTraveller	→ 7 attributes	(weak under PackageBooking)
Insurance	→ 10 attributes	(1:1 → Booking)
CancellationRefund	→ 7 attributes	(1:1 → Booking)
Review	→ 9 attributes	(1:1 → Booking)
PromoCode	→ 11 attributes	(1:N → Booking)
DemandTag	→ 5 attributes	(standalone cache)